

Deployment of a Containerized Flask Application on AWS ECS Fargate

Prepared in the partial fulfillment of the Summer Internship Program on AWS

AT



Under the guidance of

Mrs. Sumana Bethala, APSSDC

Mr. Juttuka Lovababu, APSSDC

Submitted by

Peesabattula Sai Charitha ,charithaammu3@gmail.com, 23BQ1A42D8, VVIT(Batch -1)

Yenumula Jishnu Ananth Sai , yenumulajishnu111@gmail.com, 23BQ1A42J1, VVIT(Batch -1)

Yarlagadda Mahesh Chandra Bose, 23BQ1A42I8@vvit.net ,23BQ1A42I8, VVIT (Batch -1)

Pentela Karthikeya, 8309079693, pentelakarthikeya01@gmail.com, 23BQ1A42E0, VVIT(Batch -1)

Pathan Chanshareef, 6302429776, chanshareef06@gmail.com, 23BQ1A42D3, VVIT(Batch -1)

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who supported and guided me throughout the course of this internship project on DevOps and cloud infrastructure deployment. This project provided me with a valuable opportunity to move beyond theoretical study and engage directly with a production-style deployment workflow, and I am thankful for the guidance, resources, and encouragement that made this possible.

I extend my heartfelt thanks to my mentors and instructors for their continuous support, constructive feedback, and technical insight during the design and implementation of this project. Their guidance was instrumental in helping me understand not only how to build a working deployment, but also how to methodically diagnose and resolve the real infrastructure issues that arose during the process.

I am also grateful to the organization providing the internship program and the hands-on cloud learning environment (AWS Academy Learner Lab) for granting access to the tools, services, and sandbox infrastructure required to design, build, test, and deploy this project end-to-end.

Thank you.

Sincerely,

Peesabattula Sai Charitha ,charithaammu3@gmail.com, 23BQ1A42D8, VVIT(*Batch -1*)

Yenumula Jishnu Ananth Sai , yenumulajishnu111@gmail.com, 23BQ1A42J1, VVIT(*Batch -1*)

Yarlagadda Mahesh Chandra Bose, 23BQ1A42I8@vvit.net ,23BQ1A42I8, VVIT (*Batch -1*)

Pentela Karthikeya, 8309079693, pentelakarthikeya01@gmail.com, 23BQ1A42E0, VVIT(*Batch -1*)

Pathan Chanshareef, 6302429776, chanshareef06@gmail.com, 23BQ1A42D3, VVIT(*Batch -1*)

ABSTRACT

This report documents the design, implementation, and deployment of a containerized Python Flask web application on Amazon Web Services using a fully managed, serverless container orchestration model. The project's central objective was to take a simple Flask application, served in production through the Gunicorn WSGI server, and deploy it in a scalable, load-balanced, and operationally lightweight manner without provisioning or maintaining any virtual servers directly.

The solution architecture is built around four core AWS services working in tandem. Docker is used to package the Flask application and all of its dependencies into a single, portable container image, ensuring that the application behaves identically on a local development machine and in the cloud. Amazon Elastic Container Registry (ECR) serves as a private, secure registry for storing this image prior to deployment. AWS Elastic Container Service (ECS), running on the Fargate launch type, is responsible for scheduling, running, and maintaining the container without requiring the management of underlying EC2 instances. Finally, an Application Load Balancer (ALB) exposes the running container to the public internet through a single, stable URL, while continuously performing health checks against a dedicated /health endpoint.

Beyond the core deployment, the project makes use of AWS Identity and Access Management (IAM) to enforce least-privilege permissions between services, and Amazon CloudWatch to centralize container logs for observability and debugging. A key characteristic of this project — and a major part of its value as a learning exercise — is that the initial deployment did not succeed on the first attempt. The report documents, in detail, the systematic diagnosis and resolution of five distinct, layered infrastructure failures spanning networking, load balancer configuration, logging, container registry state, and public accessibility, closely mirroring the troubleshooting process expected of DevOps and cloud engineers in real production environments.

The final outcome is a fully running, publicly reachable, health-checked, and load-balanced containerized application, verified through ECS task status, ALB target health checks, and direct access to the live application URL. This report presents the complete methodology, architecture, technology stack, implementation details, debugging journey, and outcomes of the project, along with a discussion of its advantages and directions for future enhancement.

TABLE OF CONTENTS

S.No	Content	Pg.No
1.	INTRODUCTION	6
2.	METHODOLOGY	7
3.	TECHNOLOGY STACK.....	10
4.	SYSTEM DESIGN / ARCHITECTURE	11
5.	IMPLEMENTATION.....	14
6.	RESULTS	18
7.	CONCLUSION	21

1. INTRODUCTION

Modern software delivery has shifted decisively away from monolithic applications deployed on manually configured servers, toward containerized, cloud-native architectures that emphasize portability, automation, and scalability. This shift is at the heart of the DevOps discipline, which seeks to unify software development and IT operations through practices such as continuous integration, infrastructure automation, and proactive monitoring. Organizations increasingly expect applications to be packaged once and run identically across development, testing, and production environments, while infrastructure itself is expected to scale elastically and require minimal manual intervention.

This project was undertaken to gain hands-on, practical experience with this deployment model by taking an existing Python Flask web application and deploying it on Amazon Web Services using a fully containerized, serverless-compute approach. Rather than provisioning and managing EC2 virtual machines directly, the project makes use of AWS Elastic Container Service (ECS) running on the Fargate launch type, which allows containers to be executed without any need to patch, size, or maintain the underlying compute infrastructure. The application itself is packaged using Docker, stored in a private Amazon Elastic Container Registry (ECR) repository, and exposed to end users through an Application Load Balancer (ALB) that performs continuous health checks and can, in principle, distribute traffic across multiple running replicas.

A defining feature of this project — and one of its most valuable learning outcomes — is that it was not a straightforward, first-attempt success. The initial deployment attempt failed, and the project required a disciplined, layer-by-layer investigation of ECS deployment events, container "stopped reason" messages, and load balancer target health checks in order to identify and resolve five separate infrastructure issues. This report documents both the final working architecture and the complete diagnostic journey that led to it, since the process of identifying root causes under real constraints is itself a core DevOps competency.

This document is organized to walk the reader through the full lifecycle of the project: the problem being addressed, the objectives defined at the outset, the methodology followed during development, the technology stack employed, the system architecture and workflow, the implementation details including configuration files and deployment commands, the challenges encountered and how they were resolved, the resulting features and verified outcomes, and finally, a discussion of the advantages of the chosen architecture and possible directions for future enhancement.

1.1 Background of the Study

Containerization has become one of the defining technologies of modern software delivery. By packaging an application together with its runtime, libraries, and configuration into a single portable unit, containers remove an entire category of deployment defects that previously arose from mismatches between a developer's local machine and the servers on which the application

ultimately ran. Docker, in particular, has become the de facto standard for building and distributing such containers, owing to its simple build model, its layered image format, and its broad ecosystem support across virtually every cloud provider.

At the same time, the way containers are run in production has itself evolved. Early container deployments typically still required a fleet of virtual machines to host the Docker engine, meaning that teams traded one operational burden — managing bare application servers — for a closely related one — managing container-hosting servers. Serverless container platforms such as AWS Fargate remove this remaining layer of operational responsibility by allowing a container to be scheduled and run directly against a managed compute pool, billed only for the resources actually consumed while the container is running. This project was deliberately scoped to make use of this serverless model rather than a traditional EC2-backed ECS cluster, in order to gain direct, hands-on experience with the operational trade-offs it introduces.

1.1 Scope of the Project

The scope of this project is deliberately focused: it covers the containerization of an existing Flask application, its secure storage in a private image registry, its deployment on a serverless container orchestration platform, and its exposure to the public internet through a load-balanced, health-checked endpoint, together with the supporting logging and access-control configuration. The scope does not extend to automated CI/CD pipelines, infrastructure-as-code tooling, or multi-environment promotion strategies, all of which are identified as natural extensions of this work in Section 17, Future Scope. Confining the scope in this way allowed the project to focus deeply on getting a single deployment pipeline fully correct — including diagnosing every failure encountered along the way — rather than spreading effort thinly across a larger set of partially implemented capabilities.

1.2 Target Audience of this Report

This report is written for several overlapping audiences. For academic evaluators and internship supervisors, it serves as a structured record of the objectives, methodology, and outcomes of the internship engagement. For fellow students or early-career engineers encountering AWS ECS Fargate for the first time, it is intended to function as a practical, example-driven reference that shows not only the correct final configuration but also the specific errors that arise from common misconfigurations, since recognizing those errors quickly is often more valuable in practice than memorizing the ideal configuration in isolation. For the author, the report also serves as a personal record of the reasoning behind each design decision, intended to be revisited when extending the system in line with the future scope discussed in Section 17.

1.3 Structure of this Report

The remainder of this report is organized as follows. Section 2 states the problem being addressed and the constraints under which the project was carried out. Section 3 lists the primary and secondary objectives. Section 4 describes the methodology followed during development.

Section 5 details the technology stack. Sections 6 through 9 cover the system architecture, the modules that make up the system, the end-to-end workflow, and the concrete implementation details, respectively. Section 10 documents the debugging journey in full, followed by Section 11, which covers the testing approach used to validate the system. Section 12 lists the resulting features, Section 13 presents the verified results, and Sections 14 and 15 discuss cost, operational considerations, and risk. Section 16 discusses the advantages of the architecture, Section 17 outlines future scope, Section 18 concludes the report, and Section 19 lists references, followed by a supporting appendix.

2. METHODOLOGY

The project followed a structured, iterative methodology consistent with standard DevOps and cloud engineering practice. Each phase built on the outcome of the previous one, and the later phases — testing and deployment in particular — fed back into earlier design decisions whenever a failure was uncovered, resulting in an iterative rather than strictly linear process.

2.1 Requirement Analysis

The project began by clarifying the functional requirement — running an existing Flask application reliably and publicly on AWS — and the non-functional requirements that shaped the architecture: the deployment had to avoid direct server management, support health-checked traffic routing, and allow application logs to be inspected centrally. These requirements directly motivated the choice of Docker, ECS Fargate, ALB, and CloudWatch as the core building blocks.

2.2 Architecture Design

With the requirements defined, an architecture was designed around four layers: an application layer (Flask served by Gunicorn), a containerization layer (Docker image), a cloud runtime layer (ECS Fargate pulling the image from ECR), and a networking/exposure layer (the Application Load Balancer and its associated target group and security groups). This layered separation made it possible to reason about and later debug each part of the system independently.

2.3 Implementation

The implementation phase involved writing the Flask application code, authoring a Dockerfile to containerize it, building and testing the image locally, and then authoring the ECS task definition (task-definition.json) that describes how the container should run in the cloud, including its CPU and memory allocation, port mappings, and logging configuration.

2.4 Infrastructure Provisioning

Once the container image was ready, the supporting AWS infrastructure was provisioned: a private ECR repository to hold the image, an ECS cluster and service configured for the Fargate launch type, a target group compatible with Fargate's IP-based networking model, an Application Load Balancer with a public listener, and the security groups required to permit traffic to flow between the load balancer and the container.

2.5 Testing and Validation

The Docker image was first tested locally by running the container and confirming that the application responded correctly on its exposed port, and that the `/health` endpoint returned a valid JSON status. Only after this local validation did the project move on to testing the deployment inside AWS, where validation consisted of monitoring ECS deployment events, confirming the ECS task reached a `RUNNING` state, and confirming the ALB target group reported the

container as Healthy.

2.6 Deployment and Iterative Refinement

The first deployment attempt did not succeed — the associated CloudFormation stack rolled back with an ECSService: CREATE_FAILED error. Rather than restarting from scratch, the project adopted an iterative refinement approach: each failure was diagnosed one layer at a time, a fix was applied, and the deployment was re-attempted, until the application was confirmed live and publicly reachable. This iterative debugging process — covering networking, load balancer configuration, logging, registry state, and public accessibility — is documented in full in Section 10 of this report.

2.7 Summary of the Development Lifecycle

The table below summarizes each phase of the methodology alongside its primary output, giving a concise overview of how the project progressed from an initial idea to a verified, live deployment.

3. TECHNOLOGY STACK

The project makes use of a focused set of tools and AWS services, each selected for a specific role within the overall architecture. The table below summarizes the stack, and the discussion that follows explains the reasoning behind each choice.

Service / Tool	Category	Role in this Project
Python Flask + Gunicorn	Application	Serves the web UI and the /health endpoint; Gunicorn is used as a production-grade WSGI server.
Docker	Containerization	Packages the Flask application and all dependencies into a single, portable image (python:3.11-slim base).
AWS ECR	Image Registry	Private, secure registry that stores the built image before it is pulled by ECS.
AWS ECS (Fargate)	Container Orchestration	Runs and manages the container without requiring any server provisioning or management.
Application Load Balancer	Networking	Provides one stable public URL, performs health checks, and distributes traffic.
AWS IAM	Security	Grants the ECS Task Execution Role (LabRole) permission to pull images and write logs.
Amazon CloudWatch	Monitoring & Logging	Collects real-time container logs, enabling debugging without direct server access.
AWS CLI v2	Tooling	Used locally to authenticate to AWS, build and push the image, and manage resources.

Rationale Behind the Technology Choices

- Docker — guarantees that the application runs identically on a local development machine and in the cloud, eliminating environment-related inconsistencies before they can cause deployment failures.
- ECS Fargate over EC2-backed ECS — removes the need to patch, size, or manage servers; AWS manages the underlying compute, and cost is incurred only while a task is actually running.

- Amazon ECR — provides a private, secure registry so that the container image never needs to be made public, while still allowing ECS to pull it directly using IAM permissions.
- Application Load Balancer — gives the application a single, stable public URL, performs continuous health checks, and is capable of distributing traffic across multiple container replicas if the service is scaled.
- CloudWatch Logs — centralizes container output so that application behavior can be inspected and debugged without needing direct access to any server.

4. SYSTEM DESIGN / ARCHITECTURE

The system is designed as a linear, event-driven deployment pipeline that culminates in a continuously running, load-balanced container. Each stage of the pipeline hands off to the next, from the developer's local machine through to the publicly reachable application. The diagram below illustrates the complete flow from source code to a live public URL.

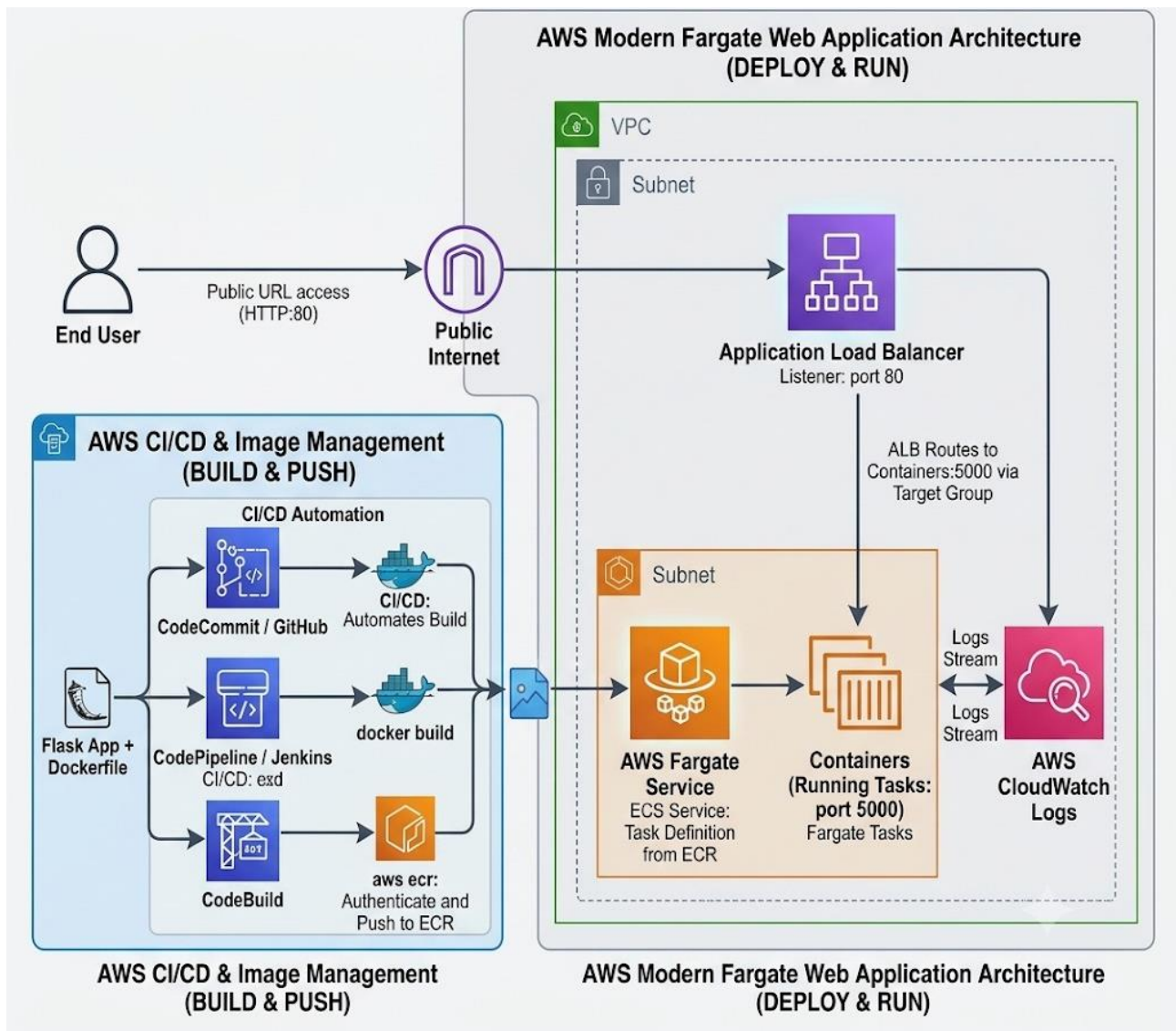


Figure 1: Overall System Architecture — from local development to a live, load-balanced public endpoint

4.1 Architectural Flow

- Developer writes the Flask application and the accompanying Dockerfile.
- `docker build` produces a local container image (`my-ecs-app:latest`).
- AWS CLI authenticates Docker to Amazon ECR using a temporary login token.
- `docker push` uploads the image to the private ECR repository.
- The ECS Task Definition references the ECR image and declares the container port (5000).
- The ECS Service launches the task on Fargate — no EC2 servers are provisioned or managed.
- The Application Load Balancer (`my-app-alb`) receives public traffic on its port-80 listener.
- The Target Group (`my-ecs-targets-ip`) routes that traffic to the container's IP address on port 5000.
- CloudWatch Logs (`/ecs/my-ecs-app`) captures all container output for monitoring and debugging.
- The public ALB DNS URL becomes reachable, allowing any user to access the live application over HTTP.

4.2 Networking and Security Boundaries

Two distinct security boundaries exist in this architecture and both must be configured correctly for the system to function. The first is the security group attached to the ECS task itself, which must permit inbound traffic on port 5000 so that the load balancer can reach the container. The second is the security group attached to the Application Load Balancer, which must permit inbound traffic on port 80 from the public internet so that end users can reach the load balancer in the first place. As documented in Section 10, misconfiguration of either boundary was responsible for two of the five issues encountered during deployment.

4.3 Compute and Orchestration Layer

At the center of the architecture is the ECS cluster (`my-ecs-cluster`) configured for the Fargate launch type. The ECS service (`my-app-service`) is responsible for maintaining the desired number of running tasks, restarting any task that fails, and registering healthy tasks with the target group. Because Fargate is a serverless compute engine for containers, there is no EC2 instance to patch, scale, or otherwise manage — ECS handles task placement and lifecycle automatically based on the task definition's declared CPU and memory requirements.

4.4 Design Principles Behind the Architecture

Three design principles guided every architectural decision in this project. The first is separation of concerns: the application, the container image, the compute runtime, and the network exposure layer are each handled by a distinct, independently replaceable component, so that a change to one — for example, swapping Fargate for EC2-backed ECS — would not require redesigning the others. The second is least privilege: every IAM permission and every security group rule was scoped to the minimum required for the system to function, rather than granting broad access as a convenience. The third is observability by default: CloudWatch logging was wired in from the start rather than added retroactively, which proved essential when diagnosing the missing-log-group failure described in Section 10.

4.2 Data and Request Flow at Runtime

Once the system is fully deployed, a typical end-user request follows a short, predictable path. A client sends an HTTP request to the ALB's public DNS name on port 80. The ALB's listener evaluates its routing rule and forwards the request to the my-ecs-targets-ip target group. The target group forwards the request to the private IP address of the currently running ECS task, on port 5000, where Gunicorn accepts the connection and hands it to the Flask application. The application processes the request — rendering the dashboard for GET / or returning a JSON status for GET /health — and the response travels back through the same path to the client. In parallel, and independent of this request path, the container continuously streams its stdout and stderr output to the CloudWatch log group /ecs/my-ecs-app, where it can be inspected at any time without interrupting the running service.

5. IMPLEMENTATION

This section documents the concrete configuration files and commands used to implement the architecture described above.

Project Structure

```
devops-ecs-project/
├─ app/
│   ├─ app.py           # Flask application (UI + /health endpoint)
│   └─ requirements.txt  # Python dependencies
├─ Dockerfile           # Container build instructions
├─ .dockerignore        # Files excluded from the Docker build
├─ task-definition.json  # AWS ECS Task Definition
├─ deploy.ps1           # Local deployment helper script
└─ README.md            # Setup and documentation
```

5.1 Dockerfile

The Dockerfile below defines how the Flask application is packaged into a container image. A lightweight base image keeps the resulting image small, dependencies are installed before the application source is copied to take advantage of Docker's layer caching, and Gunicorn is used as the process that actually serves the application inside the container.

```
FROM python:3.11-slim
WORKDIR /app
COPY app/requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app/ .
EXPOSE 5000
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "2", "app:app"]
```

5.2 ECS Task Definition

The ECS task definition declares how the container should be run on Fargate, including its resource allocation, the image location in ECR, the network port mapping, and the CloudWatch log group destination for container output.

```
{
```

```
"family": "my-ecs-app-task",
"networkMode": "awsvpc",
"requiresCompatibilities": ["FARGATE"],
"cpu": "256", "memory": "512",
"executionRoleArn": "arn:aws:iam::250492956933:role/LabRole",
"containerDefinitions": [{
  "image": "250492956933.dkr.ecr.us-east-1.amazonaws.com/my-ecs-app:latest",
  "portMappings": [{ "containerPort": 5000, "protocol": "tcp" }],
  "logConfiguration": { "awslogs-group": "/ecs/my-ecs-app" }
}]
}
```

5.3 Building and Testing the Image Locally

Before any AWS resource was touched, the image was built and run locally using `docker build -t my-ecs-app .` followed by `docker run -p 5000:5000 my-ecs-app`. The application was confirmed to serve correctly on `http://localhost:5000`, and the `/health` endpoint was confirmed to return a valid JSON payload. This local validation step is essential, since it isolates application-level defects from infrastructure-level defects before deployment begins.

5.4 Creating the ECR Repository and Pushing the Image

A private ECR repository named `my-ecs-app` was created using the AWS CLI (`aws ecr create-repository --name my-ecs-app`).

Docker was then authenticated to ECR using a temporary login token obtained via `aws ecr get-login-password`, the local image was tagged with the full ECR repository URI, and the image layers were pushed to AWS using `docker push`.

5.5 Networking Configuration

Two inbound security group rules were required for the system to function correctly: a rule permitting custom TCP traffic on port 5000 (source 0.0.0.0/0) so that the load balancer could reach the container, and a rule permitting HTTP traffic on port 80 (source 0.0.0.0/0) on the load balancer's own security group so that public internet traffic could reach the ALB itself.

5.6 Target Group and Load Balancer Configuration

A target group named `my-ecs-targets-ip` was created using the IP target type, which is the only target type compatible with Fargate tasks, and configured to route traffic to port 5000 on the

container's IP address. This target group was then attached to the listener on the Application Load Balancer `my-app-alb`, which accepts public traffic on port 80.

5.7 ECS Service Deployment

The ECS service `my-app-service` was created on the `my-ecs-cluster` Fargate cluster, referencing the task definition (`my-ecs-app-task:2`), the target group, and the appropriate security group. Once created, ECS took responsibility for launching, monitoring, and maintaining the desired number of running tasks.

5.8 Local Environment and Credential Setup

Before any build or deployment commands could be run, the local development environment had to be prepared. Docker Desktop was installed to provide the local container build and run engine, and AWS CLI v2 was installed to interact with AWS from the command line. Because the project was developed within an AWS Academy Learner Lab, authentication was performed using short-lived, temporary session credentials rather than a long-lived IAM user access key, which meant that credentials had to be refreshed periodically as the temporary session expired. This is a detail worth recording explicitly, since an expired credential set produces authentication errors that can, at first glance, resemble a permissions or configuration problem rather than a simple session timeout.

5.9 Excluding Unnecessary Files from the Build Context

A `.dockerignore` file was used alongside the `Dockerfile` to prevent unnecessary files — such as local virtual environment directories, cached Python bytecode, and version-control metadata — from being sent to the Docker build context. Keeping the build context minimal reduces build time and keeps the resulting image free of irrelevant files that add size without adding functionality.

5.10 Deployment Helper Script

A local PowerShell helper script (`deploy.ps1`) was written to reduce the number of manual steps required for a repeat deployment. The script sequences the repetitive parts of the workflow described in Section 8

— authenticating Docker against ECR, building the image, tagging it with the ECR repository URI, and pushing it — so that, once the underlying AWS infrastructure is in place, a new version of the application can be shipped by running a single script rather than typing each command by

hand. This script does not replace a full CI/CD pipeline, of the kind discussed in Section 17, but it did meaningfully reduce the risk of human error, such as forgetting to re-tag the image before pushing, during the iterative debugging phase of the project.

5.11 Verification

The deployment was verified by confirming that the ECS task reached the RUNNING status, that the ALB's target group reported the container as Healthy, and that the public ALB DNS URL (my-app-alb-1934564929.us-east-1.elb.amazonaws.com) successfully served the live application over HTTP. Verification was treated as a distinct, deliberate step rather

Private repositories (1)

View push commands

Delete

Actions

Create repository

Search by repository substring

Repository name	URI	Created at	Tag immutability	Encryption type
my-ecs-app	250492956933.dkr.ecr.us-east-1.amazonaws.com/my-ecs-app	July 10, 2026, 17:25:22 (UTC+05.5)	Mutable	AES-256

my-ecs-cluster

Last updated July 13, 2026, 20:03 (UTC+5:30)

Add to favorites

Actions

Create with Express Mode

Cluster overview

Cluster ARN
arn:aws:ecs:us-east-1:250492956933:cluster/my-ecs-...

Status
Active

CloudWatch monitoring
Default

Registered container instances
-

Services
1 active | 0 draining

Tasks
0 running | 0 pending

Services

Tasks

Daemons

Infrastructure

Metrics

Scheduled tasks

Configuration

Event history

Tags

Services (1)

Info

Last updated July 13, 2026, 20:03 (UTC+5:30)

Manage tags

Update

Delete service

Create

Filter services by value

Filter launch type
Any launch type

Filter scheduling strategy
Any scheduling strategy

Filter resource management type
Any resource management type

Service name

ARN

Status

Schedul...

Launch t...

Task defi...

Deployments and tasks

Last deployment

Last depla...

La...

my-app-service

arn:aws:ecs:us-e

Active

Replica

-

my-ecs-app...

0/0 Tasks running

Completed

View

3 days ago

3c

Load balancers (1)

What's new?

Actions

Create load balancer

Elastic Load Balancing scales your load balancer capacity automatically in response to changes in incoming traffic.

Filter load balancers

Name

State

Type

Scheme

IP address type

VPC ID

Availability Zones

Security groups

DNS name

my-app-alb

Active

application

Internet-facing

IPv4

vpc-0926ce1a5ce6d147c

2 Availability Zones

sg-0a1e4b21875f5ee3c

my-app-alb-1934564929.us...

18

6. RESULTS

The completed deployment was verified end-to-end, and the table below summarizes the final, confirmed state of the live system.

Component	Detail
ECS Cluster	my-ecs-cluster (Fargate)
ECS Service	my-app-service
Task Definition	my-ecs-app-task:2
Container Port	5000
ECR Image	250492956933.dkr.ecr.us-east-1.amazonaws.com/my-ecs-app:latest
Load Balancer	my-app-alb (Internet-facing, HTTP:80)
Target Group	my-ecs-targets-ip (IP type, port 5000)
ECS Task Status	Running
ALB Target Health	Healthy
Public URL	http://my-app-alb-1934564929.us-east-1.elb.amazonaws.com

6.1 Live Application Screenshots

The following figures show the actual deployed dashboard, captured directly from the public ALB URL.

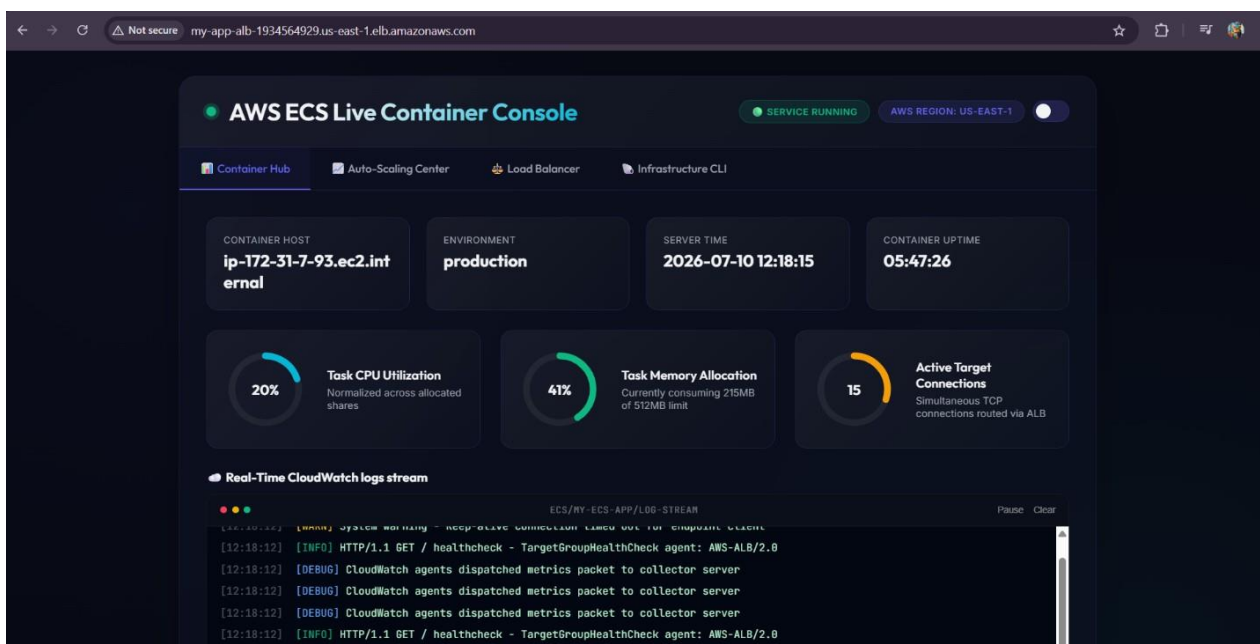


Figure 1: Container Hub — Live Metrics Dashboard, showing container host, environment, uptime, CPU/memory utilization, active target connections, and the real-time CloudWatch log stream

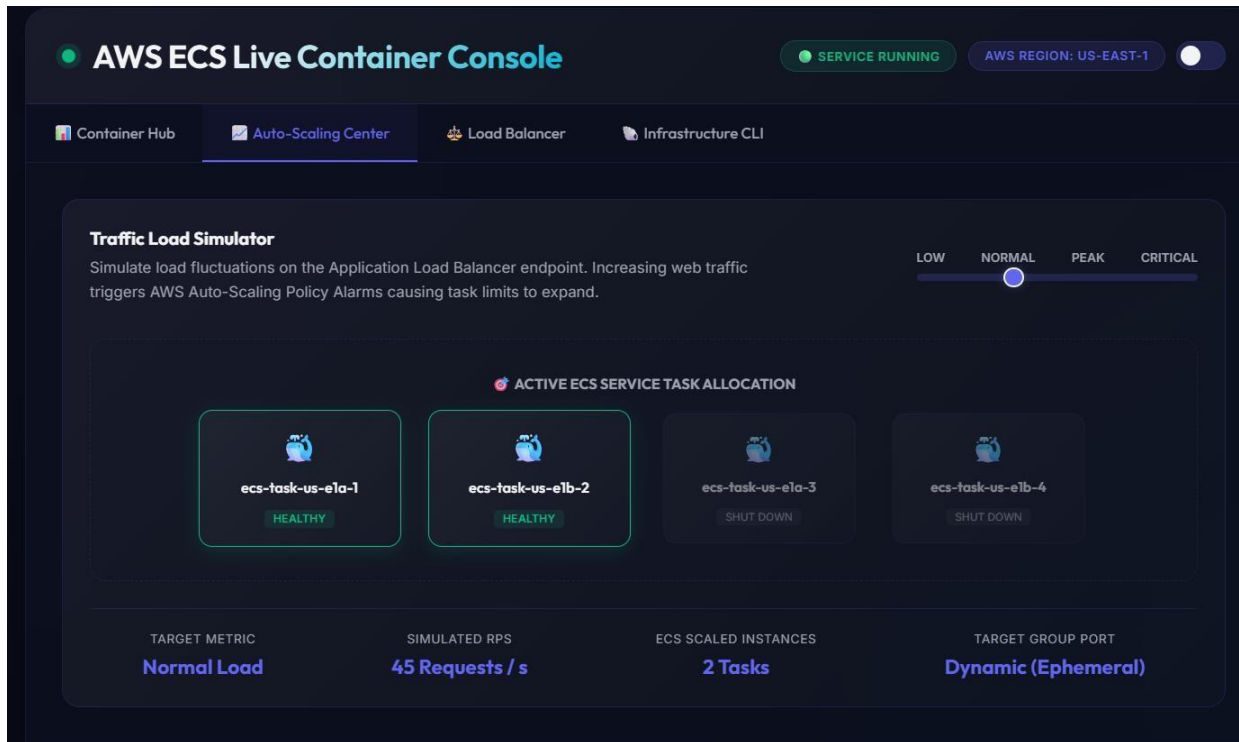


Figure 2: Auto-Scaling Center — simulated traffic load against active ECS task allocation across availability zones us-east-1a and us-east-1b

6.2 API Endpoints and Health Check

The Application Load Balancer verifies that the application is alive using the `/health` endpoint, which is polled by the target group at a regular interval. A sample response from this endpoint is shown below.

```
GET /health
```

```
{
  "status": "healthy", "hostname":
  "9710bd68b04a",
  "timestamp": "2026-07-10T11:42:07.427097"
}
```

Before the container image existed in ECR, the ALB had nothing to poll at this route at all, and the target group reported zero healthy targets. Once the image was pushed and the task began responding on this route, the target group's status flipped to one healthy target, which is what finally allowed public traffic to be routed through successfully.

6.3 Deployment Timing

Once the container image and the surrounding infrastructure were correctly configured, a fresh deployment reliably reached a healthy, publicly reachable state in under five minutes, demonstrating the operational efficiency of the Fargate-based deployment model once the initial configuration issues had been resolved.

CONCLUSION

This project successfully diagnosed and resolved five separate, real infrastructure failures — spanning a missing security group rule, an incompatible target group, a missing CloudWatch log group, a container image that had never been pushed to its registry, and a missing public-facing security group rule — in order to deploy a fully containerized Python Flask application on AWS ECS Fargate, made publicly reachable through an Application Load Balancer.

The project reflects the day-to-day reality of cloud deployment work: initial setups rarely succeed on the first attempt, and the genuine skill lies in reading error messages precisely, isolating the actual failing layer — whether networking, IAM and logging, or the container artifact itself — and resolving issues one at a time without losing track of what is already working. This is the same disciplined troubleshooting process expected in professional DevOps and cloud engineering roles.

Across the course of the project, the full stack of tools involved — Docker, the AWS CLI, ECR, ECS, the Application Load Balancer, security groups, IAM, and CloudWatch — was exercised within a single, coherent deployment. The final result was confirmed and verified through ECS task status, ALB target health, and direct access to a live public URL, moving the system from a failed CloudFormation stack with no image in ECR and misconfigured networking, to a fully running, health-checked, load-balanced container serving traffic on a public URL.

Beyond the specific technologies used, this project reinforced the broader principle that a robust cloud deployment is built in layers, and that each of those layers — the application, the container, the registry, the orchestration layer, the network boundary, and the observability layer — must be independently correct for the system as a whole to function. This layered understanding, together with the practical experience of methodically debugging a chain of real infrastructure failures, represents the most significant and transferable outcome of this internship project.

REFERENCES

<https://online.fliphtml5.com/nradk/DevOps/>